

Extensión de Andromeda a *gravitational lensing*

Métricas de lente, plano fuente y composición observacional

David Bamba
Universidad Nacional de Colombia

23 de abril de 2026

Resumen

Este documento registra la extensión del motor de *ray tracing* de Andromeda al régimen de *gravitational lensing*. A partir del núcleo numérico ya existente para agujeros negros (integrador Numba, paralelización con **prange**, eventos, detector) se añaden (i) métricas de lente en régimen de campo débil (SIS y SIE), (ii) una clase **SourcePlane** con perfiles de luz 2D compilados (Gaussiano y Sérsic), (iii) un nuevo modo **lensing** en el *dispatcher* de trazado que intersecta los rayos escapados con el plano fuente y (iv) una capa de composición visual (galaxias de fondo, *starfield*, ruido fotográfico, *stretch asinh*) para producir imágenes comparables a observaciones HST. Se documentan las hipótesis físicas, el contrato de interfaz entre módulos, los ejemplos implementados y las limitaciones conocidas.

Índice

1. Motivación y alcance	2
2. Métricas de lente en campo débil	2
2.1. Forma general del <i>line element</i>	2
2.2. Singular Isothermal Sphere (SIS)	2
2.3. Singular Isothermal Ellipsoid (SIE)	2
2.4. Contrato de interfaz	3
3. Plano fuente y perfiles de luz	3
3.1. Clase SourcePlane	3
3.2. Perfiles 2D compilados	3
4. Modo lensing del <i>dispatcher</i>	4
4.1. Geometría del detector	5
5. Composición de imagen observacional	5
6. Pipeline completo	7
7. Ejemplos implementados	7
8. Hipótesis físicas y validez	8
9. Qué está validado y qué falta validar	9
10. Resumen de archivos añadidos / modificados	9
11. Siguiendo pasos	10

1. Motivación y alcance

El núcleo de Andromeda fue diseñado para trazar geodésicas nulas en fondos de Kerr y Schwarzschild (incluyendo variantes MOG y *scalar-hair*) y producir imágenes del disco de acreción. Toda la maquinaria numérica —integrador adaptativo con detección de eventos, paralelización, detector en el plano imagen— es **agnóstica a la métrica**: cualquier espaciotiempo que exponga los *hooks* Numba (`_rhs_nb`, `_metric_nb`, `_omega_nb`) puede ser trazado sin tocar el *hot path*.

La extensión a *gravitational lensing* aprovecha esa neutralidad: se añaden métricas de lente siguiendo el mismo contrato y se introduce un nuevo tipo de objeto (`SourcePlane`) que reemplaza al disco de acreción como destino de los rayos. Todo el código numérico existente (integrador, eventos, paralelización) se reutiliza sin modificación.

Lo que NO se documenta aquí. Los detalles del integrador Numba, el *dispatcher* de eventos, la paralelización con `prange` y el contrato de los *hooks* métricos ya están cubiertos en `numba_refactor.tex`. Aquí solo se describe lo *añadido* para lensing.

2. Métricas de lente en campo débil

2.1. Forma general del *line element*

Las métricas implementadas parten de la forma isotrópica en campo débil

$$ds^2 = -A(r, \theta)^2 dt^2 + B(r, \theta)^2 (dr^2 + r^2 d\theta^2 + r^2 \sin^2 \theta d\phi^2), \quad (1)$$

con $A = 1 + \Phi$, $B = 1 - \Phi$ y $\Phi \ll 1$. Esto no es una solución exacta de las ecuaciones de Einstein; es la forma canónica del lensing gravitacional a primer orden en Φ , que reproduce correctamente la deflexión de la luz $\alpha = 2\Delta\Phi$ pero descarta términos no-lineales en Φ .

La unidad geometrizada es $G = c = 1$. No se introduce aún cosmología FRW: las distancias D_L , D_{LS} se fijan a mano en unidades de M .

2.2. Singular Isothermal Sphere (SIS)

Archivo: `scr/lens_metrics/sis.py`.

Potencial esféricamente simétrico:

$$\Phi(r) = 2\sigma_v^2 \ln\left(\frac{r}{r_{\text{ref}}}\right), \quad (2)$$

que reproduce el campo de una esfera isoterma singular con dispersión de velocidades σ_v (en unidades de c). La deflexión resultante es constante en b (parámetro de impacto):

$$\alpha_{\text{SIS}} = 4\pi\sigma_v^2, \quad (3)$$

y produce un radio de Einstein $b_E = 4\pi\sigma_v^2 D_{LS}$.

2.3. Singular Isothermal Ellipsoid (SIE)

Archivo: `scr/lens_metrics/sie.py`.

Generalización elipsoidal del SIS:

$$\Phi(r, \theta) = 2\sigma_v^2 \ln\left(\frac{\xi}{r_{\text{ref}}}\right), \quad \xi = r \sqrt{\sin^2 \theta + \frac{\cos^2 \theta}{q_{\text{ax}}^2}}. \quad (4)$$

El parámetro $q_{\text{ax}} \in (0, 1]$ es el cociente de semiejes: $q_{\text{ax}} = 1$ recupera el SIS, $q_{\text{ax}} < 1$ corresponde a un elipsoide oblato respecto al eje z .

Las derivadas parciales (usadas por el *hook* geodésico) son

$$\partial_r \Phi = \frac{2\sigma_v^2}{r}, \quad (5)$$

$$\partial_\theta \Phi = 2\sigma_v^2 \frac{\sin \theta \cos \theta (1 - 1/q_{ax}^2)}{\sin^2 \theta + \cos^2 \theta / q_{ax}^2}. \quad (6)$$

Se preserva la axisimetría en ϕ (el momento angular k_ϕ se conserva), por lo que un rayo que parte en el plano ecuatorial permanece allí.

2.4. Contrato de interfaz

Cada `LensMetric` expone exactamente los mismos atributos que `BlackHole` (ver `numba_refactor.tex`):

- `_rhs_nb(q)` — RHS geodésico compilado,
- `_metric_nb(x)` — componentes de $g_{\mu\nu}$,
- `_omega_nb(...)` — velocidad angular nula (*dummy* en lensing, requerido por el *dispatcher*),
- `EH` — pseudo-horizonte numérico (`r_min` ~ 10^{-2} actúa como corte del potencial logarítmico divergente).

Esto garantiza que `parallel.trace_*` y el integrador no distinguen entre un rayo en Kerr y un rayo en SIE: el mismo `solve_ivp` interno.

3. Plano fuente y perfiles de luz

3.1. Clase `SourcePlane`

Archivo: `scr/common/lens_image.py`.

```
class SourcePlane:
    def __init__(self, D_LS, profile):
        self.D_LS = float(D_LS) # distancia lente-fuente
        self.profile = profile # light profile 2D
        # duck-type para el dispatcher (no se usan en modo lensing)
        self._r_tbl = np.zeros(2, dtype=np.float64)
        self._I_tbl = np.zeros(2, dtype=np.float64)
        self.in_edge = 0.0
        self.out_edge = 0.0
```

El plano se sitúa perpendicular al eje óptico en $x = -D_{LS}$. El *duck-type* con atributos heredados de `thin_disk.structure` permite que el *dispatcher* de `parallel.py` acepte un `SourcePlane` donde esperaba una estructura de disco, sin ramificación adicional.

3.2. Perfiles 2D compilados

Archivo: `scr/sources/light_profiles.py`.

Cada perfil empaqueta sus parámetros en un `float64[8]` y expone una etiqueta entera `_kind`. Un único dispatcher Numba evalúa el brillo en (x, y) :

```
@njit(cache=True)
def _eval_profile_nb(xs, ys, kind, params):
    if kind == 0: # Gaussian
        ...
    elif kind == 1: # Sersic
        ...
    return 0.0
```

Gaussiano.

$$I(x, y) = I_0 \exp \left[-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2} \right]. \quad (7)$$

Sérsic elíptico.

$$I(x, y) = I_e \exp \left[-b_n \left(\left(\frac{R}{R_e} \right)^{1/n} - 1 \right) \right], \quad R = \sqrt{x_r^2 + (y_r/q)^2}, \quad (8)$$

con (x_r, y_r) rotado por el ángulo de posición ϕ alrededor de (x_0, y_0) , $q = 1 - \varepsilon$ (axial ratio), y b_n aproximado por la expansión de Ciotti & Bertin (1999)

$$b_n \simeq 2n - \frac{1}{3} + \frac{4}{405n} + \frac{46}{25515n^2}. \quad (9)$$

$n = 1$ corresponde a un disco exponencial; $n = 4$ a un perfil de Vaucouleurs típico de elípticas.

Añadir un perfil nuevo. Basta con (i) definir una nueva etiqueta `KIND_X`, (ii) añadir una rama en `_eval_profile_nb` y (iii) crear una clase con `_kind` y `_params` como atributos. Ningún otro módulo necesita cambio.

4. Modo lensing del *dispatcher*

El módulo `scr/common/parallel.py` acepta cuatro modos:

Código	Modo
0	<code>no_doppler</code>
1	<code>doppler</code>
2	<code>shadow</code>
3	<code>lensing</code> (nuevo)

En modo `lensing`:

1. El integrador traza el rayo *backwards* desde el pixel del detector hasta que se dispara el evento `escape` ($r > r_{\text{escape}}$).
2. A partir del estado final $(x_{\text{esc}}, k_{\text{esc}}^\mu)$ se extrapola en línea recta hasta el plano fuente: el rayo sale del campo gravitatorio y viaja en vacío. Se resuelve

$$x(\lambda) = -D_{LS}, \quad (10)$$

para λ y se obtiene $(y_{\text{src}}, z_{\text{src}})$.

3. Se evalúa el brillo $I = \text{_eval_profile_nb}(y_{\text{src}}, z_{\text{src}}, \text{kind}, \text{params})$ y se asigna al pixel.

`LensImage` (subclase de `Image`) es azúcar sintáctico para fijar `mode="lensing"` al llamar `create_image()`.

```
class LensImage(Image):
    def create_image(self, n_workers=None, chunksize=None):
        self._trace(mode="lensing",
                    n_workers=n_workers,
                    chunksize=chunksize)
```

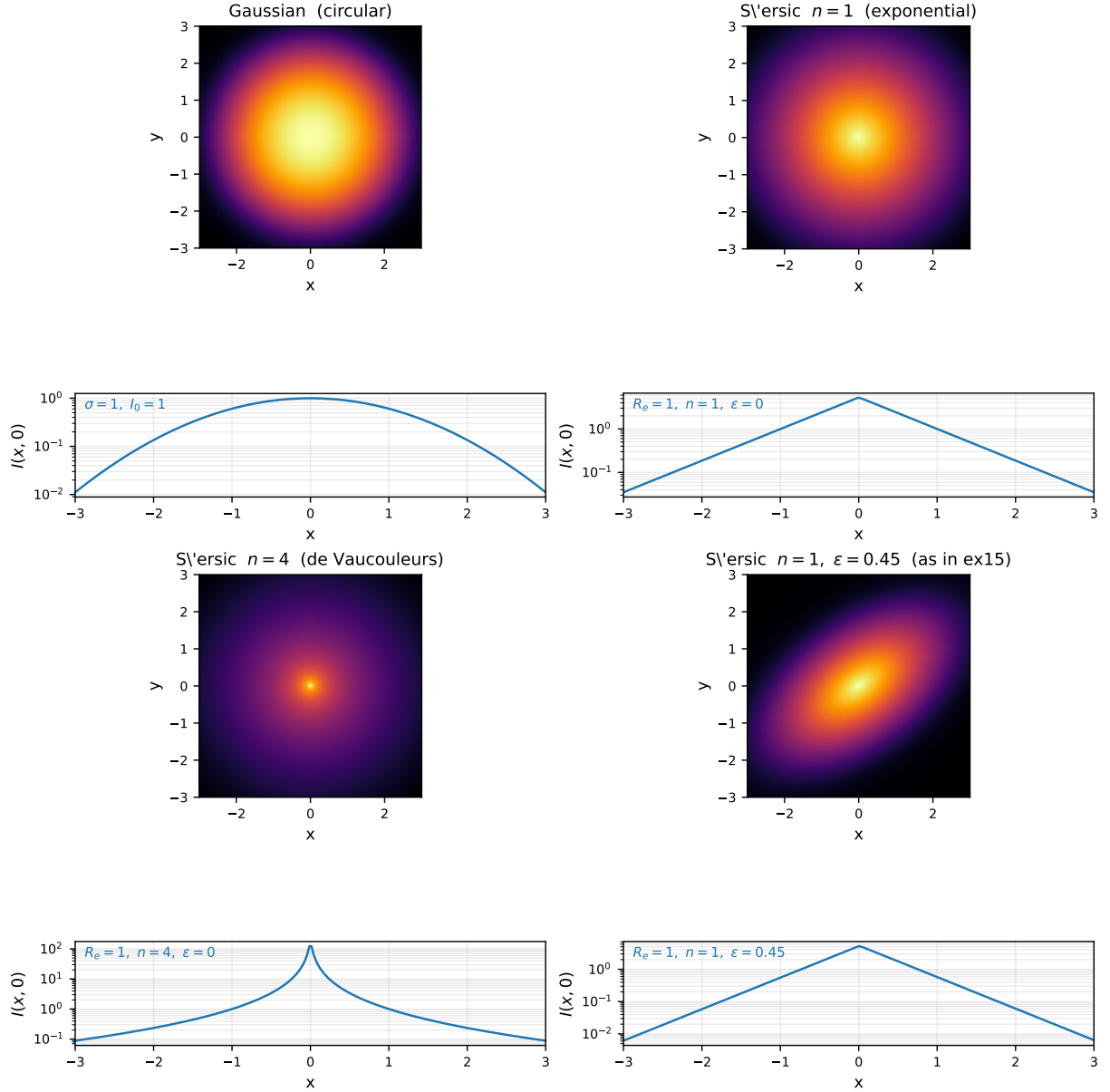


Figura 1: Perfiles de luz 2D compilados (`scr/sources/light_profiles.py`). Cada panel muestra el mapa de brillo $I(x,y)$ con *stretch* asinh y, debajo, un corte unidimensional a $y = 0$ en escala log- y . La Gaussiana es la referencia circular; el Sérsic $n = 1$ reproduce un disco exponencial; el Sérsic $n = 4$ (de Vaucouleurs) concentra brillo hacia el centro con alas extendidas; la variante con $\varepsilon = 0,45$ es la fuente usada en `ex15` para producir arcos asimétricos. Generado por `docs/figs/gen_profiles.py`.

4.1. Geometría del detector

El detector ya existente (`scr/detectors/image_plane.py`) se usa sin modificación. Para lensing conviene fijar $\iota = \pi/2$ (proyección paralela, observador en el infinito), de forma que cada pixel sea un rayo con el mismo vector $\hat{k}$ pero diferente parámetro de impacto $b = \sqrt{\alpha^2 + \beta^2}$.

5. Composición de imagen observacional

El contenido anterior basta para producir un *arco bruto* del rayo lensado. Para obtener imágenes con aspecto de observación real se añadió una capa de post-proceso en `scr/common/visual.py`:

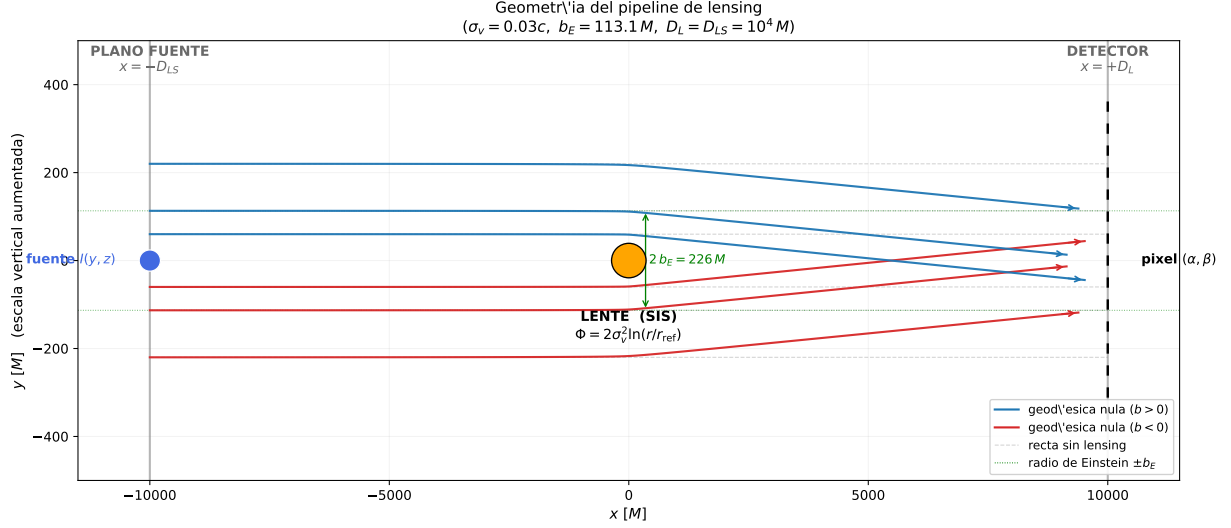


Figura 2: Geometría del pipeline de lensing. El plano fuente (izquierda, $x = -D_{LS}$) contiene la distribución de brillo $I(y, z)$; la lente (centro, naranja) curva las geodésicas nulas; el detector (derecha, $x = +D_L$) registra un pixel (α, β) por cada rayo trazado *hacia atrás*. Las trayectorias azules y rojas corresponden a geodésicas nulas reales (SIS, $\sigma_v = 0,03 c$); las líneas grises discontinuas son el contrafactual sin lensing; las dos imágenes a $\pm b_E$ forman el anillo de Einstein. Generado por docs/figs/gen_geometry.py.

project_profile_on_detector evalúa directamente un perfil 2D en el plano imagen, sin *ray tracing*. Se usa para la luz propia de la galaxia-lente (que no se lensa a sí misma) y para galaxias de fondo proyectadas.

add_starfield genera N estrellas puntuales con PSF Gaussiana, flujos log-uniformes y posiciones aleatorias.

add_photographic_noise aplica ruido Poisson (escalado por `poisson_scale`) más ruido Gaussiano de lectura. No modela *cosmic rays* ni *dark current*.

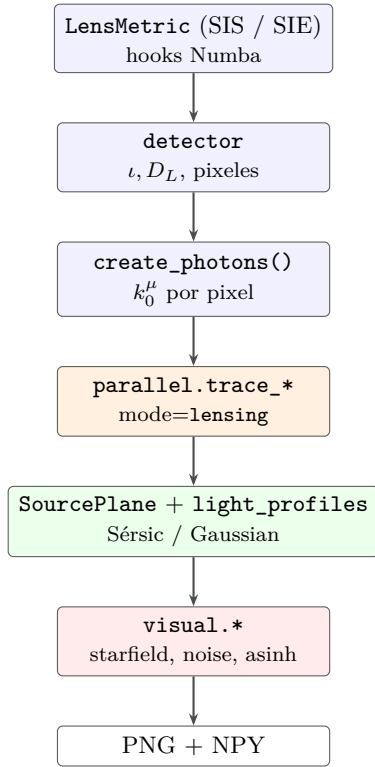
compose_rgb combina capas monocromáticas en RGB aplicando colores prefijados por componente.

asinh_stretch *stretch* percepción-lineal al estilo HST:

$$I_{\text{out}} = \frac{\text{asinh}(I/s)}{\text{asinh}(I_{\text{máx}}/s)}, \quad (11)$$

con *softening* s y máximo tomado del percentil 99.7 de la luminancia.

6. Pipeline completo



7. Ejemplos implementados

Todos los ejemplos viven en la raíz del repositorio y guardan sus salidas en `images/lensing/` y `images_data/lensing/`.

Ejemplo	Qué valida / muestra
ex9.einstein_ring	Schwarzschild puntual + fuente Gaussiana en el eje $\Rightarrow$ anillo de Einstein. Se verifica $b_{\text{ring}} = 2\sqrt{M D_{LS}}$.
ex10.cosmic_horseshoe	SIS + Sérsic desplazado + composición RGB con <i>starfield</i> suave. Produce un arco tipo “Cosmic Horseshoe” (LRG 3-757).
ex11.lensing_trajectory	Trazado directo de geodésicas nulas en SIS; diagrama con ángulos de deflexión y parámetros de impacto.
ex12.kerr_lensing	Kerr ($a = 0,9$) vs Schwarzschild lado a lado; visualiza la asimetría por <i>frame-dragging</i> en régimen de campo fuerte.
ex13.einstein_cross	SIS vs SIE ($q = 0,55$): anillo simétrico contra configuración de imágenes múltiples (cruz de Einstein).
ex14.realistic_lensing	Figura capa de realismo observacional: múltiples fuentes lensadas, galaxias de fondo no-lensadas, <i>starfield</i> , ruido fotográfico.
ex15.broken_ring_lensing	SIE + fuente primaria fuertemente elíptica ($\varepsilon = 0,45$); arcos asimétricos con “puentes” entre imágenes múltiples.

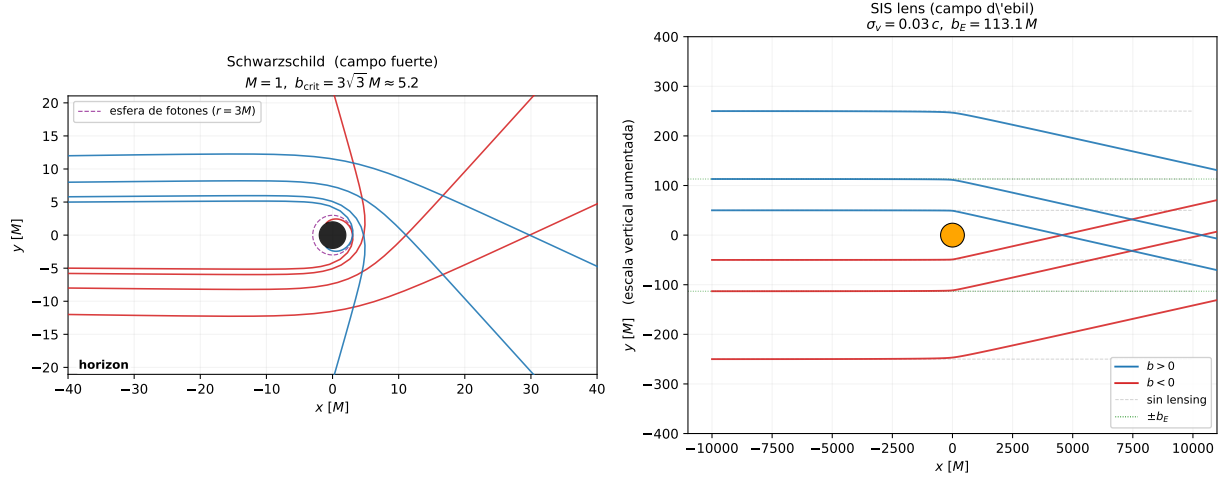


Figura 3: Mismo integrador, dos regímenes. Izquierda: Schwarzschild ($M = 1$), régimen de campo fuerte; los rayos con parámetro de impacto cerca de $b_{\text{crit}} = 3\sqrt{3}M$ rodean la esfera de fotones y algunos terminan cayendo al horizonte. Derecha: lente SIS galáctico ($\sigma_v = 0,03c$); los rayos se deflexan suavemente en un régimen de campo débil $|\Phi| \ll 1$ y todos escapan. Ambos casos usan el mismo integrador (RK45/DOP853) y el mismo sistema de eventos, cambiando únicamente el objeto métrica. Generado por `docs/figs/gen_trajectory_comparison.py`.

8. Hipótesis físicas y validez

- **Campo débil.** La métrica (1) es estrictamente válida para $|\Phi| \ll 1$. En los ejemplos se usa $\sigma_v \sim 0,028c$ (velocidades infladas respecto a galaxias reales, $\sim 200 \text{ km/s} \approx 10^{-3}c$) solo para obtener radios de Einstein visualmente representativos en unidades geometrizadas. La deflexión angular reproducida sigue siendo correcta a primer orden, pero el régimen no es estrictamente “weak” cuando se fuerzan estos valores.
- **Axisimetría.** Todas las métricas de lente conservan k_ϕ . Lentes no-axisimétricas generales (p.ej. con *external shear* en dirección arbitraria) requieren romper esta simetría y no están implementadas.
- **Un único plano fuente.** No hay *multi-plane lensing*: la intersección se hace con un solo SourcePlane a distancia D_{LS} fija.
- **Plano fuente plano.** La fuente no se extiende en *redshift*: todo el perfil vive en $x = -D_{LS}$ exactamente.
- **Cosmología ausente.** No se han introducido distancias angulares de diámetro FRW; D_L y D_{LS} son parámetros libres en unidades de M . Esto impide convertir a arcsec o kpc.
- **Galaxias de fondo no lensadas.** En los composites (ex10, ex14, ex15) las galaxias de fondo se proyectan directamente; en la realidad también sufren convergencia y *shear*, aunque en régimen débil.
- **Óptica del detector.** No hay PSF de Hubble ni patrón de difracción. La resolución está limitada solo por el muestreo angular del plano imagen.
- **Ruido.** Modelo simple Poisson + Gaussiano. No se modelan *dark current*, *cosmic rays*, *flat-field residuals* ni *charge transfer inefficiency*.
- **Colores.** Arbitrarios, no derivados de SED por *redshift* por filtro; son una convención visual.

9. Qué está validado y qué falta validar

Validado numéricamente.

- Radio de Einstein de Schwarzschild $b_E = 2\sqrt{M D_{LS}}$ (ex9).
- Deflexión constante en SIS $\alpha = 4\pi\sigma_v^2$ (verificación en ex11).
- Hamilton-constraint en cada paso del integrador (`Image.verify_Hamiltonian()`, heredado del trazado BH).

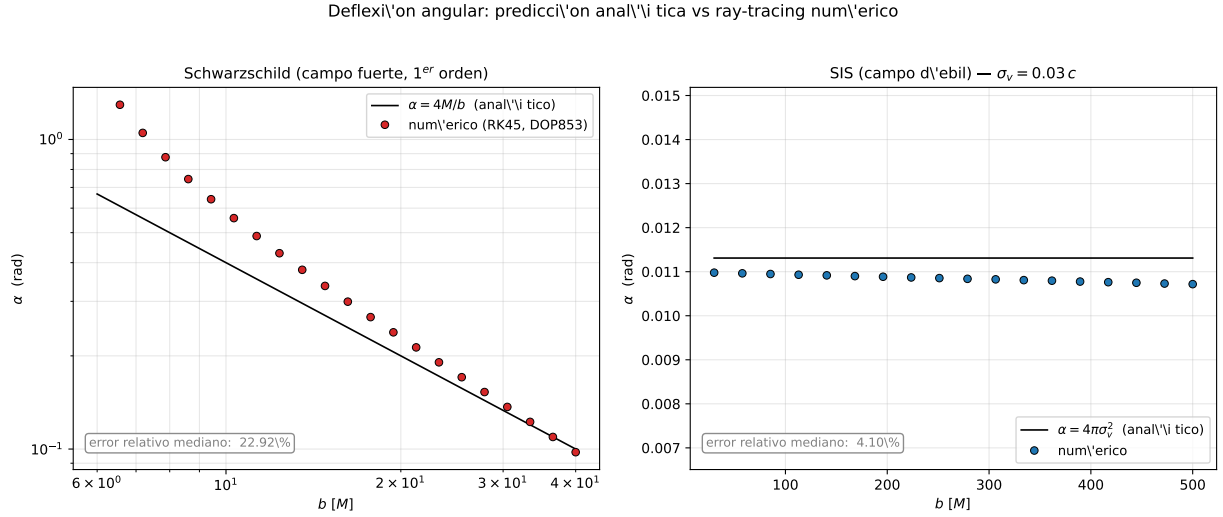


Figura 4: Validación cuantitativa de la deflexión angular. Izquierda: en Schwarzschild la deflexión $\alpha(b)$ medida por el ray-tracer (puntos rojos) sigue la predicción analítica a primer orden $\alpha = 4M/b$ (línea negra); se evitan parámetros de impacto cerca de $b_{\text{crit}} = 3\sqrt{3}M$ donde los términos no-lineales dominan. Derecha: en SIS la deflexión es independiente del parámetro de impacto ($\alpha = 4\pi\sigma_v^2$, línea negra) y los puntos numéricos (azules) reproducen el plateau. El error relativo mediano se muestra en cada panel. Generado por `docs/figs/gen_deflection.py`.

Pendiente de validación (propuesta).

- **Test de topología de imágenes múltiples** (*caustic validation*): con SIE fijo, variar solo el *offset* de la fuente para reproducir las transiciones $4 \rightarrow 3 \rightarrow 2$ imágenes al cruzar las cáusticas tangencial y radial. Si el kernel numérico reproduce las cuatro topologías con un único parámetro, el trazado en lensing está validado más allá de la simple posición del anillo.
- **Magnificación**: recuperar $\mu = 1/\det(A)$ por diferenciación numérica del mapa $\beta(\theta)$ y comparar con fórmulas analíticas para SIS/SIE (no implementado aún).

10. Resumen de archivos añadidos / modificados

Archivo	Rol
<code>scr/lens_metrics/sis.py</code>	Métrica SIS (nuevo).
<code>scr/lens_metrics/sie.py</code>	Métrica SIE (nuevo).
<code>scr/sources/light_profiles.py</code>	Gaussian + Sérsic + dispatcher Numba (nuevo).
<code>scr/common/lens_image.py</code>	SourcePlane y LensImage (nuevo).
<code>scr/common/visual.py</code>	<i>starfield</i> , ruido, RGB, <i>asinh stretch</i> (nuevo).
<code>scr/common/parallel.py</code>	Añadido modo <code>lensing</code> al <i>dispatcher</i> (modificado).
<code>ex9 - ex15</code>	Ejemplos de validación y composición observacional (nuevo).

Ninguno de los archivos del núcleo numérico BH (`integrator.py`, `_solvers.py`, `_numba_kernels.py`) fue modificado: toda la extensión es aditiva.

11. Siguiendo pasos

La hoja de ruta para aproximar aún más las imágenes a observaciones reales está descrita en `LENSING.md` y se resume en:

1. Introducir cosmología FRW (D_L , D_S , D_{LS} por *redshift*; conversión a arcsec / kpc).
2. Añadir métricas adicionales: NFW, PEMD/EPL, *external convergence* y *shear*.
3. Calcular curvas críticas, cáusticas, mapas de magnificación y *time-delay*.
4. Convolución con PSF real (HST F814W/F160W) y modelo de ruido observacional completo.
5. Fuentes extendidas no paramétricas (imágenes reales como perfil de fuente).